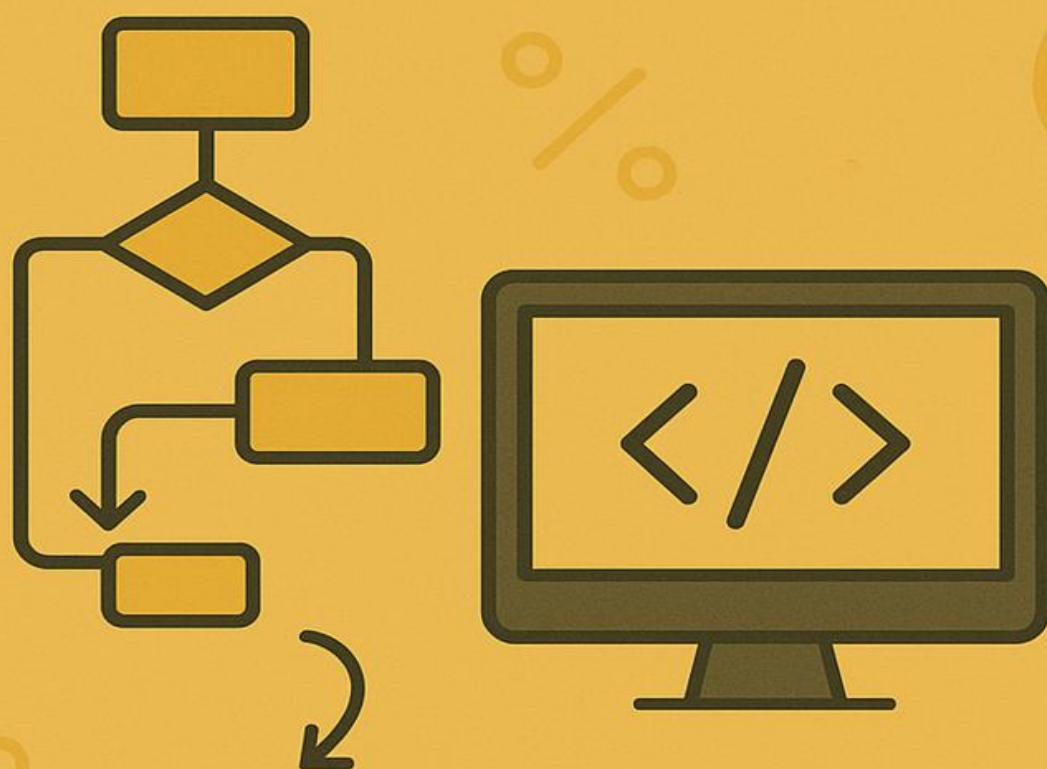


APOSTILA:

LÓGICA DE PROGRAMAÇÃO

A BASE PARA A RESOLUÇÃO
DE PROBLEMAS



INSTRUTOR:
DJALMA BATISTA
BARBOSA JUNIOR

Sumário

PARTE I - FUNDAMENTOS ESSENCIAIS

- **Capítulo 1: O Ponto de Partida: O que é Lógica de Programação?**
 - Pensando como uma máquina: Algoritmos
 - Abstração: O superpoder para resolver problemas
 - Por que aprender Lógica de Programação?
- **Capítulo 2: Fundamentos do Software**
 - 2.1. Definição e Evolução
 - 2.2. Tipos e Características de Software
 - 2.3. O Ciclo de Vida do Software: Do início ao fim

PARTE II - CONSTRUINDO OS PRIMEIROS ALGORITMOS

- **Capítulo 3: Ferramentas do Programador: Variáveis e Representações**
 - 3.1. Representando Algoritmos: Fluxogramas e Pseudocódigo
 - 3.2. Guardando Informações: Variáveis e Tipos de Dados
 - 3.3. Comunicação com o Usuário: Comandos de Entrada e Saída
 - *Exercícios Práticos do Capítulo 3*
- **Capítulo 4: A Matemática do Código: Expressões e Operadores**
 - 4.1. Expressões Aritméticas
 - 4.2. Expressões Relacionais
 - 4.3. Expressões Lógicas
 - *Exercícios Práticos do Capítulo 4*
- **Capítulo 5: Tomando Decisões: Estruturas de Controle Condicionais**
 - 5.1. O "Se" da questão: Estrutura **Se-Então**
 - 5.2. O plano B: Estrutura **Se-Então-Senão**
 - 5.3. Múltiplas Escolhas: **Escolha-Caso**
 - *Exercícios Práticos do Capítulo 5*
- **Capítulo 6: Repetindo Tarefas: Estruturas de Repetição**
 - 6.1. Repetição com Teste no Início: **Enquanto**
 - 6.2. Repetição com Teste no Final: **Repita-Até**
 - 6.3. Repetição com Contador: **Para**
 - *Exercícios Práticos do Capítulo 6*

PARTE III - ORGANIZANDO E MANIPULANDO DADOS

- **Capítulo 7: Guardando Múltiplos Dados: Vetores e Matrizes**
 - 7.1. Vetores: Organizando dados em uma linha
 - 7.2. Matrizes: Organizando dados em tabelas
 - *Exercícios Práticos do Capítulo 7*

- **Capítulo 8: Encontrando e Organizando: Busca e Ordenação**
 - 8.1. Busca Sequencial: Procurando um item na lista
 - 8.2. Ordenação de Dados: O método da Bolha (Bubble Sort)
 - *Exercícios Práticos do Capítulo 8*

PARTE IV - BOAS PRÁTICAS E PROFISSIONALISMO

- **Capítulo 9: Dividir para Conquistar: Modularização e Funções**
 - 9.1. A importância da Modularização
 - 9.2. Criando blocos de código reutilizáveis: Funções e Procedimentos
 - *Exercícios Práticos do Capítulo 9*
- **Capítulo 10: Escrevendo para Humanos: Legibilidade de Código**
 - 10.1. A Importância da Legibilidade
 - 10.2. Padrões de Nomenclatura e Convenções
 - 10.3. A Organização Visual: Indentação
 - 10.4. Deixando Pistas: Comentários
 - *Exercícios Práticos do Capítulo 10*
- **Capítulo 11: Leis do Mundo Digital: Legislação Autoral**
 - 11.1. Propriedade Intelectual: Sua criação tem valor
 - 11.2. Licenciamento de Software: Como seu código pode ser usado
- **Capítulo 12: Saúde e Bem-Estar para o Programador: Segurança no Trabalho**
 - 12.1. Normas Gerais de Segurança
 - 12.2. Ergonomia: Cuidando do seu corpo

APÊNDICES

- **Apêndice A: Gabarito dos Exercícios**
- **Apêndice B: Glossário de Termos Técnicos**

PARTE I - FUNDAMENTOS ESSENCIAIS

Capítulo 1: O Ponto de Partida: O que é Lógica de Programação?

Bem-vindo ao incrível mundo da programação! Antes de escrevermos códigos complexos e criarmos aplicativos, precisamos construir a base, o alicerce de todo o conhecimento: a **Lógica de Programação**.

Pense nela como a gramática de um idioma. Antes de escrever uma redação, você precisa entender como formar frases, usar a pontuação e organizar as ideias. A lógica é a ferramenta que nos permite "conversar" com o computador, dando instruções claras e precisas para que ele resolva problemas para nós.

1.1. Pensando como uma máquina: Algoritmos

No nosso dia a dia, seguimos algoritmos o tempo todo, mesmo sem perceber.

- **Receita de bolo?** É um algoritmo.
- **Instruções para montar um móvel?** É um algoritmo.
- **O caminho que você faz de casa para a escola?** Também é um algoritmo.

Definição: Algoritmo Um algoritmo é uma sequência finita de passos lógicos e bem definidos, que, quando executados, resolvem um problema ou executam uma tarefa.

Um algoritmo precisa ser:

- **Finito:** Ter um início e um fim.
- **Bem definido:** Cada passo deve ser claro e sem ambiguidades.
- **Eficaz:** Deve resolver o problema para o qual foi criado.

Exemplo: Algoritmo para trocar uma lâmpada queimada.

1. **INÍCIO**
2. Pegar uma escada.
3. Posicionar a escada embaixo da lâmpada.
4. Pegar uma lâmpada nova.
5. Subir na escada.
6. Girar a lâmpada queimada para a esquerda até soltar.
7. Girar a lâmpada nova para a direita até apertar.
8. Descer da escada.

1. Guardar a escada.
2. **FIM**

O objetivo da lógica de programação é treinar nossa mente para quebrar grandes problemas em pequenos passos sequenciais como este.

1.2. Abstração: O superpoder para resolver problemas

A abstração é a habilidade de focar nos aspectos essenciais de um problema, ignorando os detalhes irrelevantes. Quando você dirige um carro, você não precisa saber como a combustão ocorre no motor. Você só precisa saber usar o volante, os pedais e a marcha. Isso é abstração!

Na programação, usamos a abstração para criar modelos simplificados da realidade. Se vamos criar um sistema para uma biblioteca, não precisamos nos preocupar com a cor dos livros ou o material de que são feitos. Precisamos nos concentrar em informações essenciais como: **título**, **autor**, **código** e **status** (disponível ou emprestado).

Utilizar técnicas de abstração é o primeiro passo para transformar um problema do mundo real em algo que um computador possa entender e processar.

1.3. Por que aprender Lógica de Programação?

- **Desenvolve o raciocínio lógico:** Habilidade útil para qualquer área da vida.
- **Fundamento universal:** Os conceitos que você aprenderá aqui (variáveis, condicionais, laços) são a base de QUALQUER linguagem de programação, seja ela Python, Java, C#, JavaScript, etc.
- **Capacidade de resolver problemas:** Você aprenderá a analisar problemas de forma estruturada e a propor soluções eficazes.

Capítulo 2: Fundamentos do Software

Os algoritmos que criamos se transformam em softwares. Entender o que é software e como ele é construído nos dá uma visão completa do nosso campo de trabalho.

2.1. Definição e Evolução

Definição: Software Software é a parte lógica de um computador. É um conjunto de instruções, dados ou programas usados para operar computadores e executar tarefas específicas. Diferente do hardware (a parte física), o software é intangível.

A evolução do software é fascinante. Começou com cartões perfurados para programar teares e máquinas de calcular, passou por linguagens de baixo nível (próximas da máquina) e chegou às linguagens de alto nível que usamos hoje, muito mais próximas da linguagem humana.

2.2. Tipos e Características de Software

Podemos classificar os softwares em algumas categorias principais:

- **Software Básico:** É o software essencial para o funcionamento do hardware. O exemplo mais famoso é o **Sistema Operacional** (Windows, Linux, macOS, Android, iOS).
- **Software Aplicativo:** São os programas que usamos para realizar tarefas específicas. Exemplos: navegadores de internet (Chrome), editores de texto (Word), planilhas eletrônicas (Excel), jogos, etc.
- **Software de Programação:** Ferramentas que os desenvolvedores usam para criar outros softwares. Inclui compiladores, depuradores e editores de código.

2.3. O Ciclo de Vida do Software: Do início ao fim

Um software não nasce pronto. Ele passa por um ciclo de vida, um processo estruturado que garante sua qualidade e eficiência.

Importância: Seguir um ciclo de vida ajuda a organizar o projeto, evitar erros, controlar custos e garantir que o produto final atenda às necessidades do cliente.

As fases mais comuns do ciclo de vida são:

1. **Análise e Requisitos:** Entender o problema que o software deve resolver. O que ele precisa fazer?
2. **Design (Projeto):** Planejar a arquitetura do software. Como ele será estruturado? Como as partes se comunicarão?
3. **Implementação (Codificação):** É aqui que a lógica de programação entra! Os desenvolvedores escrevem o código com base no design.
4. **Testes:** Verificar se o software funciona como esperado e se não possui bugs (erros).
5. **Implantação:** Instalar o software para que os usuários possam utilizá-lo.
6. **Manutenção:** Corrigir bugs que aparecem com o uso e adicionar novas funcionalidades.

PARTE II - CONSTRUINDO OS PRIMEIROS ALGORITMOS

Capítulo 3:

Ferramentas do Programador: Variáveis e Representações

Agora que entendemos os conceitos, vamos colocar a mão na massa. Precisamos de ferramentas para descrever nossos algoritmos de forma clara antes mesmo de pensar em uma linguagem de programação específica.

3.1. Representando Algoritmos: Fluxogramas e Pseudocódigo

Existem duas formas principais de representar um algoritmo: uma visual e outra escrita.

A) Fluxograma

Um fluxograma é uma representação gráfica de um algoritmo. Ele usa símbolos padronizados para representar diferentes ações e decisões, conectados por setas que indicam o fluxo da execução.

Interpretar a simbologia das representações gráficas é fundamental para entender o fluxo do algoritmo.

B) Pseudocódigo (ou Português)

É uma forma de escrever o algoritmo usando uma linguagem simples e nativa (no nosso caso, o português), que se assemelha a uma linguagem de programação real, mas sem as suas regras rígidas de sintaxe. É excelente para praticar a lógica.

Exemplo: Algoritmo para somar dois números em Pseudocódigo.

Algoritmo "SomaDoisNumeros"

// Seção de Declaração de Variáveis

Var

numero1, numero2, soma: Inteiro

Inicio

// Seção de Comandos

Escreva("Digite o primeiro número: ")

Leia(numero1)

Escreva("Digite o segundo número: ")

Leia(numero2)

soma <- numero1 + numero2

Escreva("A soma dos dois números é: ", soma)

FimAlgoritmo

3.2. Guardando Informações: Variáveis e Tipos de Dados

Um programa precisa armazenar informações temporariamente para poder trabalhar com elas. Para isso, usamos as **variáveis**.

Definição: Variável Uma variável é um espaço na memória do computador reservado para armazenar um dado. Ela possui um nome (identificador) e um tipo.

Pense em uma variável como uma caixa etiquetada. A etiqueta é o **nome** da variável, e o tipo de coisa que você pode guardar dentro dela é o **tipo de dado**.

Tipos de Dados Primitivos:

- **Inteiro:** Números inteiros, sem casas decimais (ex: 10, -5, 1988).
- **Real (ou Ponto Flutuante):** Números que podem ter casas decimais (ex: 1.99, -45.7, 3.14).
- **Caractere (ou Literal):** Uma única letra, número ou símbolo, ou uma sequência deles (texto). Geralmente, fica entre aspas (ex: "Maria", "Rua A, nº 123", 'c').
- **Lógico (ou Booleano):** Armazena apenas dois valores possíveis: **Verdadeiro** ou **Falso**.

Identificar as estruturas de dados corretas para cada informação é crucial para a construção de um algoritmo eficiente.

3.3. Comunicação com o Usuário: Comandos de Entrada e Saída

- **Comando de Saída (**Escreva**):** Exibe uma mensagem ou o valor de uma variável na tela para o usuário.
 - `Escreva("Olá, mundo!")`
 - `Escreva("O resultado é: ", resultado)`
- **Comando de Entrada (**Leia**):** Pausa a execução do programa e espera o usuário digitar um valor, que será armazenado em uma variável.
 - `Leia(nomeUsuario)`
 - `Leia(idade)`

Exercícios Práticos do Capítulo 3

1. **Calculadora de Média:** Crie o fluxograma e o pseudocódigo para um algoritmo que leia duas notas de um aluno, calcule a média aritmética e exiba o resultado.
2. **Conversor de Moeda:** Elabore um algoritmo (em pseudocódigo) que leia um valor em Reais (R\$) e a cotação do Dólar, e em seguida, exiba o valor correspondente em Dólares.
3. **Dados Pessoais:** Faça um algoritmo (em pseudocódigo) que leia o nome, a idade e a altura de uma pessoa e depois exiba uma mensagem formatada na tela, como: "O(a) [Nome] tem [Idade] anos e mede [Altura]m de altura."

Capítulo 4:

A Matemática do Código: Expressões e Operadores

Os computadores são, em sua essência, calculadoras extremamente poderosas. Para instruí-los a realizar cálculos e comparações, usamos expressões e operadores.

4.1. Expressões Aritméticas

Usadas para realizar operações matemáticas.

Operador	Operação	Exemplo	Resultado (se $x=10$, $y=3$)
+	Adição	$x + y$	13
-	Subtração	$x - y$	7
*	Multiplicação	$x * y$	30
/	Divisão	x / y	3.33...
^	Potenciação	$x ^ 2$	100
MOD ou %	Módulo (Resto da divisão)	$x \text{ MOD } y$	1

4.2. Expressões Relacionais

Usadas para comparar dois valores. O resultado de uma expressão relacional é sempre um valor **Lógico** (Verdadeiro ou Falso).

Operador	Significado	Exemplo	Resultado (se a=5, b=8)
<code>==</code> ou <code>=</code>	Igual a	<code>a == b</code>	Falso
<code>!=</code> ou <code><></code>	Diferente de	<code>a != b</code>	Verdadeiro
<code>></code>	Maior que	<code>a > b</code>	Falso
<code><</code>	Menor que	<code>a < b</code>	Verdadeiro
<code>>=</code>	Maior ou igual a	<code>a >= 5</code>	Verdadeiro
<code><=</code>	Menor ou igual a	<code>b <= 8</code>	Verdadeiro

4.3. Expressões Lógicas

Usadas para combinar duas ou mais expressões relacionais. O resultado também é sempre **Verdadeiro** ou **Falso**.

Operador	Significado	Exemplo	Resultado (se media=8, presença=90)
E (AND)	Verdadeiro somente se AMBAS as condições forem verdadeiras.	(media >= 7) E (presenca >= 75)	Verdadeiro
OU (OR)	Verdadeiro se PELO MENOS UMA das condições for verdadeira.	(dia == "Sábado") OU (dia == "Domingo")	Depende
NÃO (NOT)	Inverte o valor lógico de uma condição.	NÃO (chovendo)	Se chovendo for Falso, o resultado é Verdadeiro

Utilizar expressões aritméticas, relacionais e lógicas é a base para a codificação de qualquer algoritmo que envolva cálculos ou decisões.

Exercícios Práticos do Capítulo 4

1. **Antecessor e Sucessor:** Faça um algoritmo que leia um número inteiro e mostre na tela o seu antecessor e o seu sucessor.
2. **Área do Retângulo:** Crie um algoritmo que leia a base e a altura de um retângulo e calcule e exiba a sua área ($\text{Área} = \text{base} * \text{altura}$).
3. **Verdadeiro ou Falso:** Considere as variáveis $A=10$, $B=5$, $C=8$, $D=2$. Diga se as seguintes expressões são Verdadeiras ou Falsas:
 - a) $(A * D) > C$
 - b) $(B + C) == (A)$
 - c) $(D * B) == A \text{ E } (C > A)$
 - d) $(A / D) == B \text{ OU } (B > C)$

Capítulo 5:

Tomando Decisões: Estruturas de Controle Condicionais

Raramente um programa segue um único caminho reto. Na maioria das vezes, ele precisa tomar decisões e escolher qual caminho seguir com base em certas condições. Para isso, usamos as estruturas de controle condicionais.

5.1. O "Se" da questão: Estrutura **Se-Então**

A estrutura condicional mais simples. Ela executa um bloco de código **se, e somente se**, uma condição for verdadeira.

Pseudocódigo:

Snippet de código

Se <condição> Então

// Bloco de código a ser executado

// se a condição for Verdadeira.

FimSe

Fluxograma:

Exemplo:

Snippet de código

Var

idade: Inteiro

Inicio

Escreva("Digite sua idade: ")

Leia(idade)

Se idade >= 18 Então

Escreva("Você é maior de idade.")

FimSe

FimAlgoritmo

5.2. O plano B: Estrutura Se-Então-Senão

Esta estrutura oferece um caminho alternativo. Se a condição for verdadeira, executa um bloco de código; se for falsa, executa outro.

Pseudocódigo:

Snippet de código

Se <condição> Então

// Bloco de código para condição Verdadeira.

Senão

// Bloco de código para condição Falsa.

FimSe

Fluxograma:

Exemplo:

Snippet de código

Var

media: Real

Inicio

Escreva("Digite a média final: ")

Leia(media)

Se media >= 7.0 Então

Escreva("Aluno Aprovado!")

Senão

Escreva("Aluno Reprovado.")

FimSe

FimAlgoritmo

5.3. Múltiplas Escolhas: Escolha-Caso

Quando temos que testar uma mesma variável contra vários valores possíveis, usar vários **Se-Senão** aninhados pode ser confuso. A estrutura **Escolha-Caso** é uma alternativa mais limpa.

Pseudocódigo:

Snippet de código

Escolha <variável>

Caso <valor1>

// Bloco de código para o valor1

Caso <valor2>

// Bloco de código para o valor2

// ... outros casos

OutroCaso

// Bloco de código se nenhum caso corresponder

FimEscolha

Exemplo:

Snippet de código

Var

opcao: Inteiro

Inicio

Escreva("Escolha uma opção: 1-Soma, 2-Subtração")

Leia(opcao)

Escolha opcao

Caso 1

Escreva("Você escolheu Soma.")

Caso 2

Escreva("Você escolheu Subtração.")

OutroCaso

Escreva("Opção inválida!")

FimEscolha

FimAlgoritmo

Exercícios Práticos do Capítulo 5

1. **Par ou Ímpar:** Faça um algoritmo que leia um número inteiro e diga se ele é PAR ou ÍMPAR. (Dica: um número é par se o resto da sua divisão por 2 for igual a 0).
2. **Calculadora Simples:** Melhore o algoritmo do **Escolha-Caso**. Peça ao usuário para digitar dois números e, com base na opção escolhida (1 para soma, 2 para subtração, 3 para multiplicação e 4 para divisão), realize a operação e mostre o resultado.
3. **IMC:** Crie um algoritmo que calcule o Índice de Massa Corporal ($IMC = \text{peso} / \text{altura}^2$) de uma pessoa e informe sua condição (Abaixo do peso, Peso normal, Sobrepeso, Obesidade). Pesquise os valores de referência do IMC.

Capítulo 6:

Repetindo Tarefas: Estruturas de Repetição

Muitas tarefas em programação são repetitivas. Em vez de escrever o mesmo código várias vezes, usamos estruturas de repetição (também chamadas de laços ou loops) para executar um bloco de código enquanto uma condição for verdadeira ou por um número específico de vezes.

Utilizar as estruturas de controle e repetição adequadas à lógica dos algoritmos é fundamental para criar programas eficientes e concisos.

6.1. Repetição com Teste no Início: **Enquanto**

O laço **Enquanto** (While) testa a condição **antes** de executar o bloco de código. Se a condição for falsa logo de início, o bloco nunca é executado.

Pseudocódigo:

Snippet de código

Enquanto <condição> Faça

// Bloco de código que será repetido

// enquanto a condição for Verdadeira.

FimEnquanto

Fluxograma:

Exemplo: Contar de 1 a 5.

Snippet de código

Var

contador: Inteiro

Inicio

contador <- 1

Enquanto contador <= 5 Faça

Escreva(contador)

contador <- contador + 1 // Importante: atualizar a variável de controle!

FimEnquanto

FimAlgoritmo

6.2. Repetição com Teste no Final: Repita-Até

O laço **Repita-Até** (Do-While) executa o bloco de código **pelo menos uma vez** e só depois testa a condição. A repetição continua enquanto a condição for **falsa**.

Pseudocódigo:

Snippet de código

Repita

// Bloco de código que será repetido.

Até <condição>

Exemplo: Pedir uma senha até o usuário acertar.

Snippet de código

Var

senha_digitada: Texto

Inicio

Repita

Escreva("Digite a senha: ")

Leia(senha_digitada)

Até senha_digitada == "1234"

Escreva("Acesso permitido!")

FimAlgoritmo

6.3. Repetição com Contador: Para

O laço **Para** (For) é ideal quando sabemos exatamente quantas vezes queremos que o laço se repita. Ele já inclui a inicialização, o teste e o incremento de uma variável de controle.

Pseudocódigo:

Para <variável> de <valor_inicial> ate <valor_final> Faça

// Bloco de código a ser repetido.

FimPara

Exemplo: Tabuada do 5.

Snippet de código

Var

i: Inteiro

Inicio

Escreva("Tabuada do 5:")

Para i de 1 ate 10 Faça

Escreva("5 x ", i, " = ", 5 * i)

FimPara

FimAlgoritmo

Exercícios Práticos do Capítulo 6

1. **Soma de Números:** Faça um algoritmo que peça ao usuário para digitar 5 números e, ao final, exiba a soma de todos eles. Use a estrutura **Para**.
2. **Menu de Opções:** Crie um algoritmo que exiba um menu com 3 opções e uma quarta para "Sair". O programa deve repetir a exibição do menu até que o usuário digite a opção 4. Use a estrutura **Repita-Até**.
3. **Validação de Nota:** Escreva um algoritmo que solicite uma nota entre 0 e 10. Se o usuário digitar um valor fora dessa faixa, o programa deve informar "Valor inválido" e pedir a nota novamente, até que um valor válido seja inserido. Use a estrutura **Enquanto**.

PARTE III - ORGANIZANDO E MANIPULANDO DADOS

Capítulo 7:

Guardando Múltiplos Dados: Vetores e Matrizes

Até agora, cada variável guardava apenas um valor. Mas e se precisarmos armazenar as notas de 30 alunos? Criar 30 variáveis (**nota1**, **nota2**, **nota3**...) seria inviável. Para isso, usamos estruturas de dados homogêneas, como vetores e matrizes.

7.1. Vetores: Organizando dados em uma linha

Um vetor (ou array unidimensional) é uma estrutura que armazena uma coleção de valores do **mesmo tipo** em sequência. Cada elemento do vetor é acessado por um **índice** (posição).

Declaração em Pseudocódigo: **Var nome_do_vetor: Vetor[tamanho]**
de <tipo>

Exemplo: **Var notas: Vetor[1..4] de Real** // Um vetor chamado "notas" que guarda 4 números reais.

Acessando os elementos:

- `notas[1] <- 7.5` // Atribui 7.5 à primeira posição.
- `notas[2] <- 9.0`
- `Escreva(notas[2])` // Exibe 9.0 na tela.

Exemplo de uso com laço **Para**:

Algoritmo "LeituraDeNotas"

Var

notas: Vetor[1..4] de Real

i: Inteiro

Inicio

// Loop para ler as 4 notas

Para i de 1 ate 4 Faça

Escreva("Digite a ", i, "ª nota: ")

Leia(notas[i])

FimPara

// Loop para exibir as 4 notas

Para i de 1 ate 4 Faça

Escreva("A ", i, "ª nota foi: ", notas[i])

FimPara

FimAlgoritmo

7.2. Matrizes: Organizando dados em tabelas

Uma matriz (ou array bidimensional) é uma extensão do vetor. Ela organiza os dados em linhas e colunas, como uma planilha ou um tabuleiro de xadrez. Para acessar um elemento, precisamos de dois índices: um para a linha e outro para a coluna.

Declaração em Pseudocódigo:

```
Var nome_da_matriz: Vetor[linhas, colunas] de <tipo>
```

Exemplo:

```
Var boletim: Vetor[1..2, 1..2] de Real // Uma matriz 2x2 para guardar  
as notas de 2 alunos em 2 provas.
```

Acessando os elementos:

```
boletim[1, 1] <- 8.0 // Nota do aluno 1 na prova 1. boletim[1, 2] <- 6.5  
// Nota do aluno 1 na prova 2. boletim[2, 1] <- 9.5 // Nota do aluno 2 na prova
```

Exercícios Práticos do Capítulo 7

1. **Nomes da Turma:** Crie um algoritmo que leia o nome de 5 alunos e os armazene em um vetor. Depois, exiba todos os nomes na tela.
2. **Maior Nota:** Faça um algoritmo que leia 10 notas, armazene-as em um vetor e, ao final, encontre e exiba a maior nota digitada.
3. **Tabela de Preços:** Crie um algoritmo que preencha uma matriz 3x3 com números inteiros e, em seguida, exiba a soma de todos os elementos da matriz.

Capítulo 8:

Encontrando e Organizando: Busca e Ordenação

Quando trabalhamos com grandes volumes de dados (em vetores, por exemplo), duas tarefas são muito comuns: encontrar um dado específico (busca) e colocar os dados em uma determinada ordem (ordenação).

8.1. Busca Sequencial: Procurando um item na lista

A busca sequencial (ou linear) é o método mais simples de busca. Ele percorre o vetor do início ao fim, comparando cada elemento com o valor que estamos procurando.

Lógica do Algoritmo:

1. Receber o vetor e o valor a ser buscado.
2. Usar um laço **Para** para percorrer o vetor da primeira à última posição.
3. Dentro do laço, usar um **Se** para comparar o elemento atual com o valor buscado.
4. Se encontrar, informar que o valor foi encontrado e sua posição, e parar a busca.
5. Se o laço terminar e nada for encontrado, informar que o valor não está no vetor.

Algoritmo "BuscaSequencial"

Var

numeros: Vetor[1..5] de Inteiro

num_busca, i: Inteiro

achou: Logico

Inicio

// Preenchendo o vetor (exemplo)

numeros[1] <- 10

numeros[2] <- 4

numeros[3] <- 25

numeros[4] <- 8

numeros[5] <- 13

Escreva("Qual número você deseja buscar? ")

Leia(num_busca)

achou <- Falso

Para i de 1 ate 5 Faça

Se numeros[i] == num_busca Então

achou <- Verdadeiro

Escreva("Número encontrado na posição ", i)

FimSe

FimPara

Se achou == Falso Então

Escreva("Número não encontrado.")

FimSe

FimAlgoritmo

8.2. Ordenação de Dados: O método da Bolha (Bubble Sort)

Existem muitos algoritmos de ordenação. O Bubble Sort é um dos mais simples de entender. Ele compara pares de elementos adjacentes e os troca de posição se estiverem na ordem errada. Esse processo é repetido várias vezes até que o vetor esteja completamente ordenado.

Lógica do Algoritmo:

1. Percorrer o vetor várias vezes (usando um laço dentro do outro).
2. No laço interno, comparar o elemento `vetor[i]` com `vetor[i+1]`.
3. Se `vetor[i]` for maior que `vetor[i+1]`, trocá-los de posição. Para isso, é necessária uma variável auxiliar.
4. Repetir o processo até que em uma passagem completa nenhuma troca seja feita.

Aplicar técnicas de ordenação e busca de dados é essencial para a construção de algoritmos que manipulam listas de informações de forma eficiente.

Exercícios Práticos do Capítulo 8

1. **Busca de Nomes:** Usando o vetor de nomes do exercício 1 do Capítulo 7, crie um algoritmo que peça ao usuário para digitar um nome e verifique se esse nome está presente no vetor.
2. **Ordenar Números:** Crie um algoritmo que leia 5 números inteiros, armazene-os em um vetor e, em seguida, use o método Bubble Sort para ordená-los em ordem crescente. Exiba o vetor ordenado no final.

PARTE IV - BOAS PRÁTICAS E PROFISSIONALISMO

Capítulo 9:

Dividir para Conquistar: Modularização e Funções

À medida que nossos programas crescem, o código pode se tornar longo e complexo. A **modularização** é a técnica de dividir um problema grande em partes menores, chamadas de módulos (ou sub-rotinas). Cada módulo tem uma responsabilidade específica.

9.1. A importância da Modularização

- **Organização:** O código fica mais limpo e fácil de entender.
- **Reutilização:** Um mesmo módulo pode ser chamado e usado várias vezes em diferentes partes do programa.
- **Facilita a Manutenção:** Se um erro ocorre em uma funcionalidade específica, sabemos exatamente qual módulo verificar, em vez de procurar em um código gigante.
- **Trabalho em Equipe:** Diferentes programadores podem trabalhar em módulos diferentes simultaneamente.

9.2. Criando blocos de código reutilizáveis: Funções e Procedimentos

Em pseudocódigo, representamos os módulos como **Funções** ou **Procedimentos**.

- **Procedimento (ou Sub-rotina):** Um bloco de código que executa uma tarefa, mas **não retorna** um valor.
 - Exemplo: um procedimento que apenas exibe um menu na tela.
- **Função:** Um bloco de código que executa uma tarefa e, ao final, **retorna** um valor.
 - Exemplo: uma função que recebe dois números, calcula a soma e retorna o resultado.

Exemplo de um Procedimento:

Procedimento MostraMenu()

Inicio

Escreva("=====")

Escreva("1 - Cadastrar Item ")

Escreva("2 - Listar Itens ")

Escreva("3 - Sair ")

Escreva("=====")

FimProcedimento

Exemplo de uma Função:

Funcao Soma(n1: Inteiro, n2: Inteiro) : Inteiro

Var

resultado: Inteiro

Inicio

resultado <- n1 + n2

Retorne resultado

FimFuncao

Chamando os módulos no programa principal:

Algoritmo "ProgramaPrincipal"

Var

num1, num2, total: Inteiro

Inicio

MostraMenu() // Chama o procedimento

Escreva("Digite dois números para somar:")

Leia(num1, num2)

total <- Soma(num1, num2) // Chama a função e guarda o retorno

Escreva("O resultado da soma é: ", total)

FimAlgoritmo

Exercícios Práticos do Capítulo 9

1. **Função de Média:** Crie uma **função** chamada **CalculaMedia** que receba duas notas como parâmetro e retorne a média delas. No programa principal, leia as duas notas, chame a função e exiba o resultado retornado.
2. **Procedimento de Tabuada:** Crie um **procedimento** chamado **ExibeTabuada** que receba um número como parâmetro e exiba a tabuada completa desse número (de 1 a 10). No programa principal, peça um número ao usuário e chame o procedimento.

Capítulo 10:

Escrevendo para Humanos: Legibilidade de Código

Um programa tem dois públicos: o computador, que o executa, e outros seres humanos (ou você mesmo no futuro), que precisarão lê-lo e entendê-lo. Um código que apenas funciona não é suficiente. Ele precisa ser **legível**.

10.1. A Importância da Legibilidade

- Facilita a manutenção e a correção de erros.
- Reduz o tempo necessário para entender o que o código faz.
- Permite que outras pessoas colaborem no projeto com mais facilidade.
- É um sinal de profissionalismo e cuidado.

10.2. Padrões de Nomenclatura e Convenções

A forma como você nomeia suas variáveis, funções e outros elementos do código tem um impacto enorme na legibilidade.

Regras Gerais:

- **Use nomes significativos:** Evite nomes como `x`, `y`, `a`, `coisa`. Prefira nomes que descrevam o propósito do elemento, como `idadeUsuario`, `precoFinal`, `calculaImposto`.
- **Seja consistente:** Escolha um padrão e mantenha-o em todo o projeto.

Padrões Comuns (Convenções):

- **camelCase:**
 - A primeira palavra começa com letra minúscula e as subsequentes com maiúscula. Muito usado para variáveis e funções.
 - Ex: `nomeDoCliente`, `calcularTotalNota`
- **PascalCase:**
 - Todas as palavras começam com letra maiúscula. Usado para nomes de arquivos ou classes em algumas linguagens.
 - Ex: `CalculadoraDeJuros`, `RelatorioVendas`
- **snake_case:**
 - Todas as palavras são em minúsculo, separadas por um sublinhado (`_`).
 - Ex: `nome_do_cliente`, `calcular_total_nota`
- **SCREAMING_SNAKE_CASE:**
 - Todas as palavras em maiúsculo, separadas por sublinhado. Usado para **constantes**.
 - Ex: `TAXA_DE_JUROS`, `PI`

Utilizar padrões de nomenclatura e convenções de linguagem na codificação de algoritmos é uma habilidade fundamental para o trabalho em equipe e a criação de software de qualidade.

10.3. A Organização Visual: Indentação

A indentação é o recuo de blocos de código para mostrar a hierarquia e a estrutura do algoritmo. Todo código que está "dentro" de outro (como os comandos dentro de um **Se** ou de um **Para**) deve ser indentado.

SEM INDENTAÇÃO (Ruim):

```
Se media >= 7 Então
Escreva("Aprovado")
Para i de 1 ate 3 Faça
Escreva(i)
FimPara
Senão
Escreva("Reprovado")
FimSe
```

COM INDENTAÇÃO (Bom):

```
Se media >= 7 Então
    Escreva("Aprovado")
    Para i de 1 ate 3 Faça
        Escreva(i)
    FimPara
Senão
    Escreva("Reprovado")
FimSe
```

A segunda versão é infinitamente mais fácil de ler e entender a estrutura do código.

10.4. Deixando Pistas: Comentários

Comentários são trechos de texto no código que são ignorados pelo computador, mas servem para explicar o que uma parte do algoritmo faz.

Quando comentar:

- Para explicar lógicas de negócio complexas (o "porquê" do código).
- Para documentar o que uma função faz, o que ela recebe (parâmetros) e o que ela retorna.
- Para deixar avisos ou lembretes (ex: `// TODO: Melhorar este cálculo`).

Como comentar (em pseudocódigo): Usamos `//` para comentários de uma linha.

// Função que calcula o imposto com base no salário

Funcao CalculaImposto(salario: Real) : Real

Var

taxa: Real

Inicio

// A taxa é de 15% para salários acima de 2000.00

Se salario > 2000.00 Então

taxa <- 0.15

Senão

taxa <- 0.075

FimSe

Retorne salario * taxa

FimFuncao

Identificar o padrão de nomenclatura de comentários para documentação do código fonte é essencial para a manutenção a longo prazo.

Exercícios Práticos do Capítulo 10

Refatoração de Nomes:

O código abaixo funciona, mas está mal escrito. Reescreva-o usando nomes de variáveis significativos (padrão camelCase) e para constantes (padrão SCREAMING_SNAKE_CASE).

Var n1, n2, n3, m: Real

Const val = 3

Inicio

Leia(n1, n2, n3)

m <- (n1 + n2 + n3) / val

Escreva(m)

FimAlgoritmo

Indentação e Comentários:

O código a seguir está sem indentação e sem comentários. Organize-o corretamente e adicione comentários explicando o que cada parte faz.

Algoritmo "Verificaldade"

Var i: Inteiro

Inicio

Escreva("Digite sua idade: ")

Leia(i)

Se i >= 18 Então

Escreva("Pode dirigir.")

Senão

Escreva("Não pode dirigir.")

FimSe

FimAlgoritmo

Capítulo 11:

Leis do Mundo Digital: Legislação Autoral

Assim como um livro ou uma música, o código que você escreve é uma criação intelectual e está protegido por leis. Entender esses conceitos é fundamental para a sua carreira.

11.1. Propriedade Intelectual

- **Definição: Propriedade Intelectual** Refere-se às criações da mente, como invenções, obras literárias e artísticas, símbolos, nomes e imagens usadas no comércio. No nosso caso, o software é uma obra intelectual protegida por direitos autorais.

Quem é o dono do código?

- **Se você é um funcionário:** Geralmente, o código que você escreve como parte do seu trabalho pertence à empresa que o contratou.
- **Se você é um freelancer:** O contrato de serviço deve especificar quem detém a propriedade intelectual do software final.
- **Se você cria um projeto pessoal:** Você é o detentor dos direitos autorais.

O direito autoral sobre um software garante ao seu detentor o direito exclusivo de usar, copiar, modificar e distribuir esse software.

11.2. Licenciamento de Software

Uma licença de software é um documento legal que define como um software pode ser usado e distribuído. Ela é a forma como o detentor dos direitos autorais autoriza outras pessoas a usarem sua criação.

Existem dezenas de tipos de licenças, mas podemos agrupá-las em duas grandes categorias:

A) Licenças Proprietárias:

- **Características:** São restritivas. Geralmente, o usuário compra o direito de usar o software, mas não pode ver o código-fonte, modificá-lo ou redistribuí-lo.
- **Exemplos:** Microsoft Windows, Pacote Adobe, a maioria dos jogos.

B) Licenças de Software Livre e de Código Aberto (Open Source):

- **Características:** Dão ao usuário mais liberdade. Elas garantem o direito de executar, estudar, modificar e distribuir o software (e seu código-fonte). "Livre" refere-se à liberdade, não necessariamente ao preço (embora a maioria seja gratuita).
- **Exemplos Famosos de Licenças:**
 - **GPL (GNU General Public License):** É uma licença "copyleft". Exige que qualquer software derivado de um código GPL também seja distribuído sob a licença GPL, garantindo que ele permaneça livre. Usada pelo Linux, por exemplo.
 - **MIT (Massachusetts Institute of Technology License):** É uma licença permissiva. Permite que você faça praticamente qualquer coisa com o código, inclusive usá-lo em projetos proprietários, desde que mantenha o aviso de copyright original. Usada por muitas bibliotecas famosas como React e .NET Core.
 - **Apache 2.0:** Outra licença permissiva, semelhante à MIT, mas com cláusulas explícitas sobre patentes. Usada pelo sistema Android.

Como programador, você irá consumir e produzir software. Saber qual licença usar em seus projetos pessoais e entender as licenças das ferramentas que você utiliza é uma responsabilidade profissional.

Capítulo 12:

Saúde e Bem-Estar para o Programador: Segurança no Trabalho

A carreira em tecnologia é, em grande parte, sedentária e envolve longas horas em frente a um computador. Cuidar da sua saúde e do seu ambiente de trabalho não é um luxo, é uma necessidade para garantir uma carreira longa e saudável.

12.1. Normas Gerais de Segurança

- **Instalações Elétricas:** Verifique se as tomadas e fios estão em bom estado. Evite o uso de "benjamins" (T's) sobrecarregados. Use estabilizadores ou nobreaks para proteger seus equipamentos.
- **Ambiente:** Mantenha o local de trabalho limpo, organizado e bem iluminado para evitar acidentes e cansaço visual.
- **Pausas:** Fazer pausas regulares é crucial. A cada 50 minutos de trabalho, levante-se, caminhe por 5-10 minutos, olhe para um ponto distante para descansar a visão e alongue-se.

12.2. Ergonomia

- **Definição: Ergonomia** É o estudo da adaptação do trabalho ao ser humano. O objetivo é criar um ambiente de trabalho que aumente o conforto, a segurança e a eficiência, prevenindo lesões e problemas de saúde.

Checklist Ergonômico para seu Posto de Trabalho:

1. Cadeira:

- Deve ter regulagem de altura.
- O encosto deve apoiar a curvatura da lombar.
- Seus pés devem ficar totalmente apoiados no chão (ou em um apoio para pés).

- Seus joelhos devem formar um ângulo de aproximadamente 90 graus.

2. Mesa:

- Deve ter altura suficiente para que seus cotovelos fiquem em um ângulo de 90 graus ao digitar.
- Seus antebraços devem estar relaxados e paralelos ao chão.

3. Monitor:

- O topo da tela deve estar na altura dos seus olhos ou ligeiramente abaixo.
- A distância entre seus olhos e o monitor deve ser de aproximadamente um braço esticado (50-70 cm).
- Evite reflexos de janelas ou luzes na tela.

4. Teclado e Mouse:

- Posicione-os de forma que seus punhos fiquem retos e relaxados, não dobrados para cima, para baixo ou para os lados.
- Considere o uso de teclados e mouses ergonômicos se sentir desconforto.

Prevenção de Lesões Comuns:

- **LER/DORT (Lesão por Esforço Repetitivo / Distúrbios Osteomusculares Relacionados ao Trabalho):** Causadas por movimentos repetitivos, postura inadequada e falta de pausas. Alongamentos e uma estação de trabalho ergonômica são as melhores formas de prevenção.
- **Fadiga Visual:** Causada por longos períodos olhando para a tela. Lembre-se da regra "20-20-20": a cada 20 minutos, olhe para algo a 20 pés (6 metros) de distância por 20 segundos.

Cuidar da sua saúde é o melhor investimento para a sua carreira. Um profissional saudável é mais produtivo, criativo e feliz.

APÊNDICES

Apêndice A: Gabarito dos Exercícios

Capítulo 3 - Exercício 1:

Snippet de código

Algoritmo "CalculadoraDeMedia"

Var nota1, nota2, media: Real

Inicio

Escreva("Digite a primeira nota: ")

Leia(nota1)

Escreva("Digite a segunda nota: ")

Leia(nota2)

media <- (nota1 + nota2) / 2

Escreva("A média é: ", media)

FimAlgoritmo

Capítulo 4 - Exercício 1:

Snippet de código

Algoritmo "AntecessorSucessor"

Var numero, ant, suc: Inteiro

Inicio

Escreva("Digite um número: ")

Leia(numero)

ant <- numero - 1

suc <- numero + 1

Escreva("O antecessor é ", ant, " e o sucessor é ", suc)

FimAlgoritmo

Capítulo 5 - Exercício 1:

Snippet de código

Algoritmo "ParOuImpar"

Var numero: Inteiro

Inicio

Escreva("Digite um número: ")

Leia(numero)

Se (numero MOD 2) == 0 Então

Escreva("O número é PAR.")

Senão

Escreva("O número é ÍMPAR.")

FimSe

FimAlgoritmo

Capítulo 6 - Exercício 1:

Snippet de código

Algoritmo "SomaDeNumeros"

Var i, numero, soma: Inteiro

Inicio

soma <- 0

Para i de 1 ate 5 Faça

Escreva("Digite o ", i, "º número: ")

Leia(numero)

soma <- soma + numero

FimPara

Escreva("A soma de todos os números é: ", soma)

FimAlgoritmo

Capítulo 7 - Exercício 1:

Snippet de código

Algoritmo "NomesDaTurma"

Var nomes: Vetor[1..5] de Texto

i: Inteiro

Início

Para i de 1 ate 5 Faça

Escreva("Digite o nome do ", i, "º aluno: ")

Leia(nomes[i])

FimPara

Escreva("Alunos cadastrados:")

Para i de 1 ate 5 Faça

Escreva(nomes[i])

FimPara

FimAlgoritmo

Apêndice B: Glossário de Termos Técnicos

- **Algoritmo:** Uma sequência finita de passos bem definidos para resolver um problema.
- **Abstração:** Focar nos aspectos essenciais de um problema, ignorando detalhes irrelevantes.
- **Bug:** Um erro ou falha em um programa de computador.
- **Compilador:** Um programa que traduz o código-fonte escrito em uma linguagem de programação de alto nível para uma linguagem que o computador entende (linguagem de máquina).
- **Constante:** Um valor que não pode ser alterado durante a execução do programa.
- **Fluxograma:** Uma representação gráfica de um algoritmo.
- **Hardware:** A parte física de um computador.
- **Indentação:** O recuo de blocos de código para mostrar a estrutura hierárquica.
- **Laço (Loop):** Uma estrutura que repete um bloco de código várias vezes.

- **Linguagem de Programação:** Um conjunto de regras e símbolos usados para escrever programas de computador.
- **Modularização:** A técnica de dividir um sistema complexo em partes menores e independentes (módulos).
- **Pseudocódigo:** Uma descrição de alto nível, informal e em linguagem natural de um algoritmo.
- **Software:** A parte lógica de um computador; os programas e dados.
- **Variável:** Um espaço na memória para armazenar um dado que pode mudar durante a execução do programa.
- **Vetor (Array):** Uma estrutura de dados que armazena uma coleção de elementos do mesmo tipo.